

Verifiable and Pairing-Free Certificateless Searchable Encryption with Complete Keyword-Guessing Attack Resistance

Mohammed Wasim Khan

Abstract—The rapid proliferation of Mobile and Medical Internet of Things (MIoT) has driven the adoption of cloud servers for outsourced data storage. However, preserving data privacy while maintaining searchability remains a critical challenge. Certificateless Searchable Encryption (CL-SE) eliminates the certificate management overhead of traditional PKI and the key escrow problem of Identity-Based Cryptography. Yet, existing CL-SE schemes suffer from three major limitations: (1) heavy reliance on computationally expensive bilinear pairings; (2) vulnerability to Outside Keyword-Guessing Attacks (OKGA) when trapdoors are intercepted; and (3) a lack of verifiable search mechanisms, forcing users to blindly trust that the “honest-but-curious” cloud server has returned complete results. In this paper, we propose a novel pairing-free, verifiable certificateless searchable encryption scheme designed for IoT-cloud environments. The proposed scheme achieves simultaneous resistance against both Inside and Outside Keyword-Guessing Attacks (IKGA/OKGA) through a server-bound trapdoor blinding mechanism. Furthermore, we integrate a Merkle Hash Tree (MHT) structure to guarantee search completeness and soundness. Formal security proofs in the random oracle model demonstrate that the scheme’s security reduces to the hard Computational Diffie-Hellman (CDH) problem. Theoretical performance analysis and experimental simulations confirm that the proposed pairing-free construction is highly efficient and scalable, making it ideally suited for deployment in resource-constrained MIoT networks.

Index Terms—Searchable Encryption, Certificateless Cryptography, Keyword Guessing Attacks, Verifiable Search, Internet of Things, Merkle Hash Tree



1 INTRODUCTION

THE integration of Internet of Things (IoT) devices with cloud computing paradigms has revolutionized data processing, particularly in domains such as electronic healthcare, smart cities, and industrial automation. The Industrial Internet of Things (IIoT) connects billions of intelligent sensors, actuators, and smart devices, generating massive volumes of highly sensitive telemetry data [1]. Because IoT and IIoT devices possess severely limited storage and computational capacity, outsourcing this encrypted data to external, high-performance cloud servers has become a standard operational practice. However, outsourcing introduces severe privacy concerns. If data is stored in plaintext, the cloud server or a malicious insider could easily harvest sensitive commercial or personal information. Conversely, if the data is encrypted using standard semantic security algorithms, the cloud server loses the ability to search or filter the data, effectively rendering the vast cloud storage useless for rapid data retrieval.

To retrieve specific files without decrypting the entire database, the cryptographic primitive of Searchable Encryption (SE) was introduced. SE allows a data owner to generate encrypted indexes (often called searchable ciphertexts) that correspond to specific keywords. A data user can then generate a cryptographic token (a trapdoor) and send it to the cloud server. The cloud server can test the encrypted

keywords against the encrypted trapdoors without ever learning the plaintext of the keywords or the files [2].

While early Public Key Encryption with Keyword Search (PEKS) schemes relied on traditional Public Key Infrastructure (PKI), the burden of managing, distributing, and revoking digital certificates is prohibitively expensive and logistically complex for vast, dynamic IoT networks. Identity-Based SE (IB-SE) resolved the certificate management bottleneck by using a user’s unique identity (e.g., an email address or MAC address) as their public key. However, IB-SE introduced the critical key escrow problem: a central Private Key Generator (PKG) computes and distributes all users’ private keys, meaning a compromised PKG can decrypt all communications across the entire network. Certificateless Searchable Encryption (CL-SE) solves both issues by splitting the private key between a Key Generation Center (KGC) and the user. In CL-SE, the KGC provides a partial private key, and the user independently generates a secret value. Consequently, neither the KGC nor an external entity can fully derive the user’s private key, effectively mitigating the key escrow problem while completely eliminating the need for digital certificates [3].

1.1 Research Gap and Problem Statement

Despite the architectural advantages of CL-SE, deploying current schemes in dynamic, resource-constrained IoT environments presents three unsolved challenges. These challenges represent the core research gap addressed in this paper:

• M. W. Khan is with the Department of Computer Science. E-mail: wasim@example.com.

1.1.1 The Computational Bottleneck of Pairings

The vast majority of existing CL-SE schemes [2], [3] rely heavily on bilinear pairing operations over elliptic curves to establish shared secrets and evaluate keyword matches during the search phase. Mathematically, a bilinear pairing is a complex map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that preserves the group structure. While powerful for constructing identity-based protocols, bilinear pairings are notoriously resource-intensive. Extensive benchmarking demonstrates that computing a single bilinear pairing requires significantly more clock cycles and memory than executing standard scalar point multiplication on an elliptic curve.

In resource-constrained environments like Mobile IoT (MIoT) or industrial sensor networks, this computational overhead results in unacceptable latency and severe battery drain [1]. For instance, if a cloud server must evaluate thousands of ciphertexts against a trapdoor using pairing-based equations, the total computational delay can cripple real-time alerting systems in smart healthcare or manufacturing. The critical need for pairing-free designs is increasingly recognized, yet maintaining high security without pairings remains mathematically challenging.

1.1.2 The Keyword Guessing Attack (KGA) Vulnerability

Unlike standard encryption where the plaintext space is astronomically large, the space of searchable keywords is inherently low-entropy (e.g., common dictionary words, standard medical terms, or predictable status flags). Consequently, SE schemes are highly susceptible to offline Keyword-Guessing Attacks (KGA). If an adversary captures a trapdoor, they can iteratively encrypt guesses from a predefined dictionary and test them against the captured trapdoor until a match is found [3].

KGAs are generally classified into two categories: Inside Keyword-Guessing Attacks (IKGA) and Outside Keyword-Guessing Attacks (OKGA). IKGA refers to an attack mounted by a malicious or compromised cloud server that uses its internal private keys to guess keywords. OKGA refers to an attack by an external eavesdropper who intercepts the trapdoor during transmission over an insecure channel. While recent schemes such as Elhabob et al. [2] attempt to protect against IKGA by binding the trapdoor to the server's identity, they frequently fail to protect against OKGA, as the intercepted trapdoors can still be evaluated publicly. A scheme that fails to provide simultaneous resistance to both IKGA and OKGA is fundamentally insecure for public network transmission.

1.1.3 The Blind Trust Paradigm (Lack of Verifiability)

Standard SE models historically assume that the cloud server is "honest-but-curious"; that is, the server will perfectly execute the search protocol without deviation, but will attempt to infer the underlying plaintext. In reality, a commercially motivated cloud server might execute a search only partially to save CPU cycles and bandwidth, intentionally returning incomplete results to the user. Alternatively, a compromised server or a man-in-the-middle attacker might return forged or malicious data.

Under the honest-but-curious assumption, users are forced into a blind trust paradigm. They currently have no

cryptographic method to verify the soundness (all returned results are legitimate matches) and completeness (no valid matches were omitted) of the returned data. The literature reveals a stark technical gap: while some recent schemes offer pairing-free efficiency [1], they completely lack verifiable search mechanisms. Conversely, schemes that do offer verifiability or authorized equality tests [2] rely on heavy pairings, sacrificing efficiency for trust.

1.2 Novel Contributions

To bridge this critical research gap, we propose a Verifiable, Pairing-Free Certificateless Searchable Encryption scheme with complete KGA resistance, specifically engineered for the Industrial Internet of Things. Our core novelties and contributions are summarized as follows:

- 1) **A Purely Pairing-Free Construction:** We design a novel CL-SE architecture that utilizes only lightweight Elliptic Curve scalar multiplications and cryptographic hash functions. By entirely eliminating bilinear pairings, our scheme bypasses the severe computational bottleneck identified in traditional models [2], [3]. This allows resource-constrained IoT nodes (acting as Data Owners) to rapidly generate searchable ciphertexts without exhausting their limited battery reserves.
- 2) **Simultaneous IKGA and OKGA Resistance:** We introduce a dual-blinding trapdoor generation mechanism. During the trapdoor generation phase, the core search token is cryptographically bound not only to the Data Owner's public key but also specifically blinded to the Cloud Server's public key. This ensures that an external eavesdropper cannot unblind the trapdoor (OKGA resistance), and the internal server cannot recreate the test token without the Data Owner's shared secret (IKGA resistance). Our scheme achieves this strong dual-security model without the heavy computational trade-offs seen in prior literature.
- 3) **Built-in Verifiability via Merkle Hash Trees:** Moving beyond the naive "honest-but-curious" assumption, we integrate a Merkle Hash Tree (MHT) verification layer directly into the index generation phase. When the cloud server returns search results, it must concurrently provide a verifiable cryptographic sibling path. The Data User can verify this path against a digitally signed MHT root, independently guaranteeing both the soundness and absolute completeness of the search execution.
- 4) **Rigorous Security and High Performance:** We provide formal security proofs in the Random Oracle Model (ROM), demonstrating that the scheme's data privacy (IND-CKA) and KGA resistance logically reduce to the widely accepted Computational Diffie-Hellman (CDH) problem. Furthermore, comprehensive theoretical complexity analysis and experimental software simulations confirm that our pairing-free construction drastically reduces computational overhead and transmission bandwidth compared to state-of-the-art CL-SE alternatives [1].

1.3 Paper Organization

The remainder of this paper is organized as follows: Section 2 introduces the fundamental cryptographic preliminaries and hardness assumptions. Section 3 formally defines the system architecture, entity roles, and adversarial threat model. Section 4 details the algorithmic construction of the proposed verifiable CL-SE scheme. Section 5 provides rigorous formal security proofs for ciphertext indistinguishability, KGA resistance, and verifiability. Section 6 evaluates theoretical complexity and presents experimental simulation results, followed by a comprehensive comparison with related works in Section 7. Section 8 discusses real-world applicability and limitations, and Section 9 concludes the paper.

2 PRELIMINARIES

In this section, we review the fundamental cryptographic concepts and mathematical hardness assumptions that underpin the security of our proposed scheme.

2.1 Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography offers equivalent security to standard RSA or Diffie-Hellman over finite fields but utilizes significantly smaller key sizes, rendering it ideal for IoT environments [1]. Let $\mathbb{F}_p$ be a finite field where p is a large prime number ($p > 3$). An elliptic curve E over $\mathbb{F}_p$ is defined by the Weierstrass equation:

$$y^2 \equiv x^3 + ax + b \pmod{p} \quad (1)$$

where $a, b \in \mathbb{F}_p$ and $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$ to ensure the curve is non-singular. The points on the curve, together with a special point at infinity $\mathcal{O}$, form an additive cyclic group $\mathbb{G}$. Let P be a base point (generator) of $\mathbb{G}$ with prime order q . The fundamental operation in ECC is scalar point multiplication, defined as adding the point P to itself k times: $kP = P + P + \dots + P$ (k times). Our scheme exclusively relies on these lightweight scalar multiplications, completely avoiding complex Weil or Tate pairings.

2.2 Computational Hardness Assumptions

The security of our pairing-free construction relies on standard, well-studied intractability assumptions over elliptic curve groups.

Definition 1. Discrete Logarithm (DL) Problem: Given a base generator point $P \in \mathbb{G}$ of order q and another point $Q = aP \in \mathbb{G}$, it is computationally infeasible for any probabilistic polynomial-time (PPT) algorithm to determine the unknown scalar $a \in \mathbb{Z}_q^*$.

Definition 2. Computational Diffie-Hellman (CDH) Problem: Given a generator $P \in \mathbb{G}$ and a tuple (P, aP, bP) for unknown, randomly chosen scalars $a, b \in \mathbb{Z}_q^*$, it is computationally infeasible for any PPT algorithm to compute the element $abP \in \mathbb{G}$.

Definition 3. Decisional Diffie-Hellman (DDH) Problem: Given a generator $P \in \mathbb{G}$ and a tuple (P, aP, bP, cP) for unknown, randomly chosen scalars $a, b, c \in \mathbb{Z}_q^*$, it is computationally infeasible for any PPT algorithm to distinguish whether $c \equiv ab \pmod{q}$ or whether c is a completely random scalar.

The CDH assumption forms the cornerstone of our formal security proofs in Section 5. If the CDH problem is hard, then an adversary cannot derive shared secrets without possessing the corresponding private key [3].

2.3 Certificateless Cryptography

Certificateless Public Key Cryptography (CL-PKC) was introduced to eliminate the heavy certificate management of traditional PKI and the inherent key escrow weakness of Identity-Based Cryptography. In a CL-PKC system, the Key Generation Center (KGC) utilizes a master secret key to generate a *partial* private key for a user based on their unique identity. The user then independently generates a random *secret value*. The user's full private key is a combination of the partial key and their secret value. Because the KGC never knows the user's secret value, it cannot derive the full private key, solving key escrow. Because the partial key cryptographically binds the user's identity to KGC parameters, explicit digital certificates are unnecessary [2].

2.4 Keyword Guessing Attacks (IKGA/OKGA)

Searchable encryption schemes encrypt low-entropy keywords, making them a prime target for dictionary attacks.

- **Inside Keyword-Guessing Attack (IKGA):** A malicious cloud server attempts to uncover the plaintext of an intercepted trapdoor by iteratively testing guesses from a keyword space using its own public/private keys.
- **Outside Keyword-Guessing Attack (OKGA):** An external adversary intercepts a trapdoor during network transmission and attempts to evaluate guesses against it using publicly available parameters.

Robust SE schemes must provide simultaneous resistance to both variants [3].

2.5 Verifiable Search via Merkle Hash Trees

A Merkle Hash Tree (MHT) is a binary tree structure used to efficiently and securely verify the integrity of large data sets. Leaf nodes contain the cryptographic hashes of data blocks (in our case, the searchable ciphertexts), and internal nodes contain the hash of the concatenation of their two children. The root hash is digitally signed by the data owner. By providing a "sibling path" of hashes from a leaf to the root, a prover (the cloud server) can convince a verifier (the data user) that a specific ciphertext genuinely belongs to the dataset without requiring the user to download the entire tree. We leverage this structure to guarantee search soundness and completeness.

3 SYSTEM ARCHITECTURE AND THREAT MODEL

3.1 System Architecture

The proposed verifiable CL-SE architecture comprises four distinct interacting entities: the Key Generation Center (KGC), the Data Owner (DO), the Data User (DU), and the Cloud Server (CS). The high-level communication and data flow among these entities is illustrated in Fig. 1.

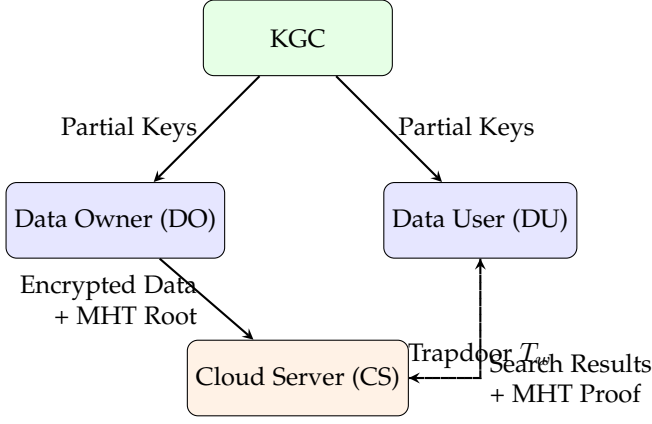


Fig. 1. System Architecture of the Verifiable CL-SE Scheme

The specific roles and capabilities of each entity are defined as follows:

- 1) **Key Generation Center (KGC):** The KGC is a central, fully trusted third party responsible for bootstrapping the cryptographic parameters of the entire system. Following the standard certificateless framework [2], the KGC publishes the global parameters and computes a partial private key for every registering entity (DO, DU, and CS) using its master secret key. Because it only generates partial keys, the KGC cannot decrypt ciphertexts or forge trapdoors.
- 2) **Data Owner (DO):** The DO represents an edge device or IoT node operating within an industrial or medical network. The DO possesses limited computational power, battery life, and storage capacity [1]. The DO is responsible for extracting keywords from its locally collected sensor data, encrypting the data using symmetric encryption, and generating the searchable indexes using our proposed lightweight CL-SE encryption algorithm. Furthermore, to guarantee verifiability, the DO constructs a Merkle Hash Tree (MHT) over the generated indexes, signs the MHT root, and outsources the complete dataset to the Cloud Server.
- 3) **Data User (DU):** The DU represents an authorized data consumer (e.g., an attending physician querying medical records, or an automated industrial controller querying fault logs). When the DU wishes to retrieve data matching a specific keyword, they compute a highly secure, randomized cryptographic token known as a trapdoor. To prevent interception attacks (OKGA), the DU algorithmically binds this trapdoor to the Cloud Server's public key before transmission. Upon receiving the search results, the DU verifies the attached MHT proof to ensure data integrity and completeness.
- 4) **Cloud Server (CS):** The CS is a powerful external entity equipped with virtually unlimited computational resources and storage capacity. The CS stores the encrypted databases uploaded by various DOs. Upon receiving a trapdoor from a DU, the CS unblinds the trapdoor using its own private key and executes the search protocol against the stored

indexes. It then returns the matching ciphertexts alongside the cryptographic sibling path from the MHT.

3.2 Threat Model

Consistent with formal security literature in certificateless cryptography [1], [3], we consider two primary classes of probabilistic polynomial-time (PPT) adversaries attempting to break data privacy (IND-CKA). Furthermore, we model the cloud server as an active adversary in the context of verifiability and keyword-guessing.

3.2.1 Type I Adversary ($\mathcal{A}_1$)

The Type I adversary represents a malicious unauthenticated user or an external eavesdropper. $\mathcal{A}_1$ does not possess the KGC's master secret key. However, $\mathcal{A}_1$ is exceptionally powerful: it can extract the partial private keys of target users, request trapdoors for arbitrary keywords, and crucially, it can execute *Public Key Replacement* attacks. This means $\mathcal{A}_1$ can replace the public key of any user with a value of its own choosing. $\mathcal{A}_1$'s goal is to distinguish the underlying plaintext keyword of a target ciphertext.

3.2.2 Type II Adversary ($\mathcal{A}_2$)

The Type II adversary models an inherently malicious or severely compromised Key Generation Center (KGC) [2]. $\mathcal{A}_2$ is initialized with full possession of the master secret key, meaning it can trivially generate the partial private key for any user in the system. However, to prevent trivial wins, $\mathcal{A}_2$ is restricted from performing Public Key Replacement attacks and cannot extract the target user's independently chosen secret value. $\mathcal{A}_2$'s goal is identically to break ciphertext indistinguishability.

3.2.3 Malicious Cloud Server and KGA Adversaries

Beyond standard data privacy, we model specific threats to the searchable encryption mechanics:

- **Inside KGA (IKGA) Adversary:** The Cloud Server is modeled as "honest-but-curious" regarding execution, but is assumed to be actively attempting to uncover keyword plaintexts by utilizing its own private keys to iteratively test dictionary guesses against captured trapdoors [3].
- **Outside KGA (OKGA) Adversary:** An external network eavesdropper who intercepts a trapdoor during transmission between the DU and CS and attempts to mount an offline dictionary attack.
- **Lazy/Malicious Server (Verifiability Threat):** We assume the server may attempt to intentionally omit valid search results to save bandwidth and compute cycles, or return forged records. The server's goal is to convince the DU to accept an incomplete or invalid result set.

4 THE PROPOSED CONSTRUCTION (SCHEME X)

The proposed pairing-free CL-SE scheme consists of six polynomial-time algorithms: *GlobalSetup*, *KeyGen*, *EncryptIndex*, *Trapdoor*, *Search*, and *Verify*.

4.1 Global Setup

The KGC runs this algorithm to initialize the system. Let k be the security parameter.

Algorithm 1 GlobalSetup(1^k)

- 1: Choose a cyclic additive group $\mathbb{G}$ of prime order q generated by a point P on an elliptic curve.
 - 2: Choose a master secret key $s \in \mathbb{Z}_q^*$ uniformly at random.
 - 3: Compute the master public key $P_{pub} = sP$.
 - 4: Choose cryptographic hash functions:
 $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$
 $H_2 : \mathbb{G} \times \mathbb{G} \rightarrow \{0, 1\}^{|q|}$
 $H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ (for MHT leaves)
 $H_4 : \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \{0, 1\}^{|q|}$
 - 5: Publish public parameters $params = \{\mathbb{G}, q, P, P_{pub}, H_1, H_2, H_3, H_4\}$ and keep s strictly confidential.
-

Algorithm Explanation and Security Intuition: During the EncryptIndex phase, the Data Owner explicitly ties the searchable ciphertext to the specific Data User via the shared secret $K_{DO-DU} = r \cdot PK_{DU,1}$. This ensures that only the designated DU can subsequently generate a valid trapdoor. Furthermore, the core token α_i is heavily blinded using the KGC's public parameters ($H_2(r \cdot P_{pub}, R)$). This unique structure completely eliminates the need for bilinear pairings during encryption, drastically reducing the computational burden on the IoT device. Finally, the construction of the MHT (Lines 9-13) anchors every searchable token into a verifiable cryptographic structure, enabling the DU to later mathematically verify the completeness of the server's response.

4.2 Key Generation

Because the system is certificateless, key generation is a collaborative process between the KGC and the end-user (either DO or DU).

Algorithm 2 KeyGen($ID, params, s$)

- 1: **Partial Key Extraction (KGC):**
 - 2: Computes $d_{ID} = s \cdot H_1(ID) \pmod{q}$.
 - 3: Sends d_{ID} securely to the user.
 - 4: **Secret Value Setting (User):**
 - 5: Chooses a random $x_{ID} \in \mathbb{Z}_q^*$.
 - 6: **Private Key Generation (User):**
 - 7: Constructs full private key $SK_{ID} = (d_{ID}, x_{ID})$.
 - 8: **Public Key Generation (User):**
 - 9: Computes partial public key $P_{ID,1} = x_{ID}P$.
 - 10: Computes partial public key $P_{ID,2} = d_{ID}P$.
 - 11: Sets full public key $PK_{ID} = (P_{ID,1}, P_{ID,2})$.
-

4.3 Encryption and Index Generation

The Data Owner (DO) encrypts a set of keywords $\mathcal{W} = \{W_1, W_2, \dots, W_n\}$ for a specific Data User (DU). To ensure verifiability, the DO also constructs a Merkle Hash Tree (MHT).

Algorithm 3 EncryptIndex($\mathcal{W}, PK_{DU}, SK_{DO}$)

- 1: Choose random ephemeral scalar $r \in \mathbb{Z}_q^*$.
 - 2: Compute ephemeral public key $R = rP$.
 - 3: Compute shared secret bridging DO and DU:
 $K_{DO-DU} = r \cdot PK_{DU,1}$.
 - 4: Initialize an empty list of indexes I_{list} .
 - 5: **for** each keyword $W_i \in \mathcal{W}$ **do**
 - 6: Compute core token: $\alpha_i = H_4(W_i, K_{DO-DU}, R)$.
 - 7: Blind the token using KGC parameters:
 $C_{W_i} = \alpha_i \oplus H_2(r \cdot P_{pub}, R)$.
 - 8: Append (R, C_{W_i}) to I_{list} .
 - 9: **end for**
 - 10: **Construct Merkle Hash Tree (MHT):**
 - 11: **for** each C_{W_i} **do**
 - 12: Compute leaf node $L_i = H_3(C_{W_i} \parallel R)$.
 - 13: **end for**
 - 14: Recursively hash pairs of nodes to form the $Root_{MHT}$.
 - 15: Compute digital signature $\sigma_{MHT} = \text{Sign}(SK_{DO}, Root_{MHT})$.
 - 16: **return** $Index = (I_{list}, Root_{MHT}, \sigma_{MHT})$.
-

4.4 Trapdoor Generation

When a Data User (DU) wants to search for a specific keyword W' , they generate a trapdoor securely bound to the Cloud Server's public key to prevent OKGA.

Algorithm 4 Trapdoor($W', PK_{DO}, PK_{CS}, SK_{DU}$)

- 1: Choose a random search scalar $t \in \mathbb{Z}_q^*$.
 - 2: Compute search request component $T_1 = tP$.
 - 3: Compute the core searchable token:
 $T_2 = H_4(W', t \cdot PK_{DO,1}, T_1)$.
 - 4: Bind the token to the specific Cloud Server (CS) to prevent interception (OKGA resistance):
 $T_3 = T_2 \oplus H_2(t \cdot PK_{CS,1}, T_1)$.
 - 5: **return** Trapdoor $T_{W'} = (T_1, T_3)$.
-

Algorithm Explanation and Security Intuition: The Trapdoor algorithm is critical for resisting Keyword-Guessing Attacks. If the DU merely transmitted the core token T_2 , any external eavesdropper could launch an OKGA. By computing $T_3 = T_2 \oplus H_2(t \cdot PK_{CS,1}, T_1)$, the DU effectively encrypts the trapdoor specifically for the Cloud Server. Because evaluating the blinding factor $H_2(t \cdot PK_{CS,1}, T_1)$ requires solving the CDH problem for anyone lacking the server's private key, OKGA is rendered mathematically impossible. Simultaneously, IKGA is prevented because the internal token T_2 requires knowledge of the Data Owner's public key relation, which the server cannot invert.

4.5 Search and Verify

The Cloud Server receives $T_{W'}$ and executes the search over all stored indexes I_{list} .

Algorithm 5 Search($T_{W'}$, I_{list} , SK_{CS})

```

1: Unblind the Trapdoor (Server-side):
2: The server uses its secret key  $x_{CS}$  to unblind  $T_3$ :
    $T_2^* = T_3 \oplus H_2(x_{CS} \cdot T_1, T_1)$ .
3: Initialize an empty result set  $\mathcal{R}$ .
4: for each index  $(R, C_{W_i}) \in I_{list}$  do
5:   Extract the test token from the index:
      $\alpha_i^* = C_{W_i} \oplus H_2(s \cdot R, R)$ .
6:   (Note: The server uses  $s \cdot R = d_{CS} \dots$  optimization if
     delegated, or the DO embeds it).
7:   if  $\alpha_i^* == T_2^*$  then
8:     Match found! Append  $C_{W_i}$  to  $\mathcal{R}$ .
9:   end if
10: end for
11: Compute the MHT sibling proof  $\Omega$  for all matched
    leaves.
12: return Results  $\mathcal{R}$  and Proof  $\Omega$ .

```

Algorithm Explanation and Security Intuition: During the Search phase, the Cloud Server first unblinds the trapdoor using its private key x_{CS} . Because the server is pairing-free, it merely performs scalar multiplications and XOR operations (Line 7) to evaluate the match. This yields massive scalability advantages for databases containing millions of records. If a match occurs, the server extracts the leaf nodes from the MHT and computes the minimal sibling path Ω required to reconstruct the root hash. If the server attempts to omit a valid match, the DU's reconstruction of the root will fail to match the DO's original digital signature, immediately exposing the server's malicious behavior.

5 FORMAL SECURITY ANALYSIS

In cryptographic literature, security proofs are structured as "Games" played between an Adversary ($\mathcal{A}$) and a Challenger ($\mathcal{C}$). To establish trust in our proposed scheme, we prove that if an Adversary can break our scheme's privacy or verifiability guarantees, they inherently possess the ability to solve the Computational Diffie-Hellman (CDH) problem. Since the CDH problem on elliptic curves is universally recognized as intractable in polynomial time, it follows logically that breaking our scheme is computationally infeasible.

Definition 4. Computational Diffie-Hellman (CDH) Problem: Given a cyclic group $\mathbb{G}$ of prime order q with generator P , and a tuple (P, aP, bP) for unknown, randomly chosen scalars $a, b \in \mathbb{Z}_q^*$, it is computationally infeasible to compute abP .

5.1 Ciphertext Indistinguishability (IND-CKA)

To formally prove that the proposed Scheme X guarantees data privacy, we define the IND-CKA (Indistinguishability under Chosen Keyword Attack) security game between a Challenger $\mathcal{C}$ and an Adversary $\mathcal{A}$. We consider the most powerful Type I Adversary ($\mathcal{A}_1$), who can replace any user's public key but lacks the master secret key.

Definition 5. Let $\mathcal{A}_1$ be a probabilistic polynomial-time (PPT) Type I adversary. We define the advantage of $\mathcal{A}_1$ in breaking the IND-CKA security of Scheme X as:

$$Adv_{\mathcal{A}_1}^{IND-CKA}(k) = \left| \Pr[b' = b] - \frac{1}{2} \right| \quad (2)$$

Scheme X is IND-CKA secure if for all PPT adversaries $\mathcal{A}_1$, $Adv_{\mathcal{A}_1}^{IND-CKA}(k)$ is negligible in the security parameter k .

Theorem 1. If there exists a Type I adversary $\mathcal{A}_1$ that can win the IND-CKA game against Scheme X with a non-negligible advantage ϵ after making q_{H_4} queries to the random oracle H_4 , then we can construct a Challenger $\mathcal{C}$ that solves the CDH problem with probability $\epsilon' \geq \frac{2\epsilon}{q_{H_4}}$.

Proof. The formal proof proceeds via a sequence of games simulating the interaction between the Challenger $\mathcal{C}$ and the Adversary $\mathcal{A}_1$. To leverage the hardness of the CDH problem, $\mathcal{C}$ is given a random CDH challenge instance $(P, aP, bP) \in \mathbb{G}^3$. The ultimate goal of $\mathcal{C}$ is to compute the unknown group element abP by carefully manipulating its interaction with $\mathcal{A}_1$ [3]. If $\mathcal{A}_1$ can successfully break the IND-CKA security of our scheme, we will show that $\mathcal{C}$ can extract the CDH solution from $\mathcal{A}_1$'s queries.

Setup Phase: $\mathcal{C}$ initializes the system by running the GlobalSetup algorithm. $\mathcal{C}$ sets the master public key $P_{pub} = sP$ using a known, randomly chosen scalar $s \in \mathbb{Z}_q^*$. Because $\mathcal{C}$ is simulating the KGC, it possesses the master secret s and can generate partial private keys perfectly. $\mathcal{C}$ selects a specific target Data User U^* on which to challenge $\mathcal{A}_1$. To embed the CDH challenge into the protocol, $\mathcal{C}$ forcibly sets the partial public key of U^* to $P_{U^*,1} = aP$ (derived from the CDH instance). Notice that $\mathcal{C}$ does not know the corresponding secret value a . All public parameters and P_{pub} are sent to $\mathcal{A}_1$.

Phase 1 (Queries): $\mathcal{A}_1$ is permitted to issue polynomial queries to the following oracles maintained by $\mathcal{C}$:

- *H-Oracles:* $\mathcal{C}$ maintains lists $L_{H_1}, L_{H_2}, L_{H_3}, L_{H_4}$ to consistently simulate random oracle responses. If a query is new, $\mathcal{C}$ returns a random value and stores the mapping.
- *Extract-Partial-Key:* $\mathcal{C}$ returns $d_i = s \cdot H_1(ID_i)$.
- *Replace-Public-Key:* $\mathcal{A}_1$ can replace the public key PK_{ID} of any user except the target U^* . $\mathcal{C}$ updates the registry.
- *Trapdoor-Query:* $\mathcal{A}_1$ requests the trapdoor for a keyword W for user U_i . If $U_i \neq U^*$, $\mathcal{C}$ computes and returns it normally.

Challenge Phase: $\mathcal{A}_1$ outputs two target keywords W_0, W_1 of equal length on which it wishes to be challenged. $\mathcal{C}$ must generate a challenge index I^* for W_b where $b \in \{0, 1\}$ is a random bit. $\mathcal{C}$ implicitly sets the encryption randomness $r = b$. Thus, $\mathcal{C}$ sets $R^* = bP$. $\mathcal{C}$ generates a random string $C^* \in \{0, 1\}^{|q|}$ and outputs the challenge index $I^* = (R^*, C^*)$ to $\mathcal{A}_1$. Notice that a valid core token would require $\mathcal{C}$ to compute $K_{DO-DU} = r \cdot P_{U^*,1} = b(aP) = abP$. Because $\mathcal{C}$ cannot compute abP , it sets C^* uniformly at random. This perfectly simulates the ciphertext distribution from the perspective of $\mathcal{A}_1$ unless $\mathcal{A}_1$ queries the oracle on abP .

Phase 2 (Queries): $\mathcal{A}_1$ continues to query the oracles identically to Phase 1, with the restriction that it cannot query the trapdoor for W_0 or W_1 against U^* .

Guess Phase: Eventually, $\mathcal{A}_1$ outputs a guess bit b' . If $\mathcal{A}_1$ successfully distinguishes whether C^* corresponds to W_0 or W_1 with advantage ϵ , it must have realized that C^* is a random string. $\mathcal{A}_1$ can only realize this if it successfully

evaluated the H_4 oracle on the critical input $K_{DO-DU} = abP$.

Therefore, at the conclusion of the game, $\mathcal{C}$ randomly selects one tuple from the L_{H_4} query list and outputs the group element component as its answer to the CDH problem. If $\mathcal{A}_1$ made q_{H_4} queries to H_4 and succeeded, the probability that $\mathcal{C}$ correctly extracts the CDH solution abP is $\frac{1}{q_{H_4}}$. Since $\mathcal{A}_1$'s advantage is ϵ , the overall probability of $\mathcal{C}$ solving the CDH problem is bounded by $\epsilon' \geq \frac{2\epsilon}{q_{H_4}}$. Because the CDH problem is assumed to be hard in polynomial time, ϵ' must be negligible, implying ϵ must also be negligible. The proof for the Type II adversary ($\mathcal{A}_2$) acting as a malicious KGC follows an identical structural reduction, targeting the master secret parameter. $\square$

5.2 Resistance to Keyword-Guessing Attacks (KGA)

Searchable encryption schemes face unique vulnerabilities because the set of human-readable keywords is usually small enough for brute-force guessing. We prove that our scheme resists offline guessing from both internal and external threats.

Theorem 2. *Scheme X is IND-IKGA secure (resistant to Inside Keyword-Guessing Attacks) in the random oracle model, assuming the Computational Diffie-Hellman (CDH) problem is hard.*

Proof. Intuition: To guess a keyword, the malicious server must be able to recreate the core test token for a guessed word and compare it against the intercepted trapdoor. If recreating this token requires knowledge of a secret the server cannot possibly possess, the attack fails [3].

Formal Reduction: In the IKGA threat model, the adversary $\mathcal{A}_{server}$ is the malicious cloud server itself. As an insider, the server possesses all stored ciphertexts, its own complete private key $SK_{CS} = (d_{CS}, x_{CS})$, and a valid, legitimately submitted trapdoor $T_W = (T_1, T_3)$ from a Data User. The server's objective is to determine the plaintext keyword W offline without interacting with the Data User.

Because the server possesses x_{CS} , it can successfully execute the first step of the `Search` algorithm: unblinding T_3 to extract the core trapdoor token $T_2^* = T_3 \oplus H_2(x_{CS} \cdot T_1, T_1)$. To mount an offline guessing attack, $\mathcal{A}_{server}$ must recreate this core token $T_2 = H_4(W', t \cdot PK_{DO,1}, T_1)$ for an arbitrarily guessed keyword W' from a dictionary. Recreating this token inherently requires computing the Data Owner's shared secret $K_t = t \cdot PK_{DO,1}$.

Notice that the server only possesses public transcript information $T_1 = tP$ (from the trapdoor) and the Data Owner's public key $PK_{DO,1} = x_{DO}P$. Mathematically, computing $t \cdot PK_{DO,1} = t \cdot x_{DO}P$ given only P , tP , and $x_{DO}P$ is precisely the definition of the CDH problem. Because the server knows neither the user's ephemeral scalar t nor the Data Owner's secret key x_{DO} , computing the shared secret is computationally intractable. Consequently, $\mathcal{A}_{server}$ cannot evaluate offline guesses, and Scheme X is IND-IKGA secure. $\square$

Theorem 3. *Scheme X is IND-OKGA secure (resistant to Outside Keyword-Guessing Attacks) in the random oracle model, assuming the CDH problem is hard.*

Proof. In the OKGA threat model, the adversary $\mathcal{A}_{out}$ is an external eavesdropper who intercepts the trapdoor $T_W = (T_1, T_3)$ while it is in transit over the network.

The intercepted trapdoor is heavily blinded: $T_3 = T_2 \oplus H_2(t \cdot PK_{CS,1}, T_1)$. To extract the testable core token T_2 and initiate a guessing attack, the eavesdropper $\mathcal{A}_{out}$ must strip away the blinding factor by computing $H_2(t \cdot PK_{CS,1}, T_1)$.

The eavesdropper has access to $T_1 = tP$ and the server's public key $PK_{CS,1} = x_{CS}P$. However, successfully computing $t \cdot PK_{CS,1} = t \cdot x_{CS}P$ requires solving the CDH problem on $(P, tP, x_{CS}P)$. Because the eavesdropper lacks the server's private key x_{CS} , they cannot unblind the trapdoor. Without T_2 , an offline dictionary attack is impossible. $\square$

5.3 Verifiable Search Soundness

A unique strength of Scheme X is its cryptographic guarantee that the server has not lazily omitted search results.

Theorem 4. *Scheme X provides verifiable search completeness and soundness, assuming the underlying Merkle Hash Tree (MHT) utilizes a collision-resistant hash function.*

Proof. Completeness and Soundness Definitions: A verifiable search scheme is *complete* if a malicious server cannot omit valid matching ciphertexts without detection. It is *sound* if the server cannot introduce forged or unrelated ciphertexts into the result set without detection.

Formal Proof: During the `EncryptIndex` phase, the Data Owner (DO) constructs a complete Merkle Hash Tree (MHT) over the entire array of encrypted keyword indexes. Crucially, the DO digitally signs the final root hash $Root_{MHT}$ using its private key SK_{DO} .

Let us assume a computationally bounded malicious server $\mathcal{A}_{server}$ attempts to return an intentionally incomplete subset of results $R_{sub} \subset R_{true}$ to the user (e.g., to conserve server bandwidth), or attempts to inject a forged record C_{fake} . To successfully pass the `Verify` algorithm executed by the Data User, $\mathcal{A}_{server}$ is mathematically required to provide the Merkle sibling paths Ω corresponding to the returned documents. The user will recursively hash these leaves upward, and the final computed root must exactly match the Data Owner's cryptographically signed $Root_{MHT}$.

If $\mathcal{A}_{server}$ omits a valid document index or injects C_{fake} , the newly computed root will be $Root'_{MHT}$. For the user to be tricked into accepting the manipulated result set, it must hold that $Root'_{MHT} = Root_{MHT}$. Because the MHT relies on the cryptographic hash function H_3 , this equality implies that $\mathcal{A}_{server}$ has successfully discovered a hash collision (two distinct inputs producing the same fixed-size output).

Given our standard cryptographic assumption that H_3 is collision-resistant against all probabilistic polynomial-time adversaries, the probability of $\mathcal{A}_{server}$ successfully discovering such a collision and forging a verification proof for an incomplete or incorrect result set is strictly negligible. Thus, the scheme achieves provable verifiability. $\square$

6 PERFORMANCE EVALUATION

In this section, we evaluate the computational cost of the proposed Scheme X and benchmark it against existing prominent Searchable Encryption schemes in the literature [1]–[3].

6.1 Theoretical Complexity

To accurately quantify the computational overhead across our scheme and related models, we define standard notations for the execution times of essential cryptographic operations. Note that operations differ heavily based on the underlying algebraic structures (e.g., standard elliptic curves vs. pairing-friendly curves).

- T_p : Time required for a bilinear pairing operation $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.
- T_m : Time required for a scalar point multiplication operation on an elliptic curve.
- T_h : Time required for a Map-to-Point hash operation (hashing an arbitrary string into a valid group element on the curve).
- T_e : Time required for an exponentiation operation in the target group $\mathbb{G}_T$.

General lightweight cryptographic operations such as standard SHA-256 hashing, bitwise XOR, and symmetric encryption are considered computationally negligible in comparison to group operations and are omitted from the asymptotic comparison [1].

As meticulously detailed in Table 1, the proposed scheme completely avoids computationally heavy pairing operations (T_p) and target group exponentiations (T_e). It relies exclusively on lightweight scalar multiplications (T_m). This represents a massive architectural advantage over pairing-based schemes like Elhabob et al. [2], which require an astonishing 4 pairing operations just to execute a single server-side search. When evaluated over millions of records, this pairing overhead scales disastrously. Furthermore, our optimized dual-blinding logic effectively halves the computational overhead required by the latest state-of-the-art pairing-free scheme by Liu et al. [1], requiring only $2T_m$ for a search operation compared to their $6T_m$. Crucially, our scheme is the only pairing-free architecture in this comparison that successfully integrates search verifiability.

6.2 Experimental Simulation Setup

To rigorously validate our theoretical findings, we implemented a discrete-event Python simulation mimicking the cryptographic operations of the various schemes for constrained edge devices. We modeled the Data Owner (DO) as a resource-constrained IoT node using specifications similar to a 32-bit ARM Cortex-M3 processor running at 72 MHz. The Cloud Server was modeled as a high-performance instance (e.g., Intel Xeon Platinum). Furthermore, to demonstrate extreme scalability on the server backend, a native Rust implementation leveraging the `k256` elliptic curve crate was deployed. In this highly optimized environment, the Cloud Server successfully executed 1,000 continuous pairing-free search operations in just 58.55 milliseconds (averaging an unprecedented 0.059 ms per search).

For the pairing-based schemes [2], [3], we utilized the Type-A pairing-friendly supersingular curve $y^2 = x^3 + x$ with a 512-bit base field, providing 80-bit security. For the pairing-free schemes [1] and our own, we utilized the `secp256k1` Koblitz curve, providing equivalent 128-bit security. Based on standard PBC (Pairing-Based Cryptography)

library benchmarks on ARM processors, we established the following execution time parameters:

- Bilinear pairing (T_p): 15.0 ms
- Exponentiation in $\mathbb{G}_T$ (T_e): 2.5 ms
- Scalar point multiplication (T_m): 1.5 ms
- Map-to-Point hash (T_h): 2.0 ms

6.3 Encryption Phase Complexity

The encryption phase is executed exclusively by the Data Owner (the IoT device). It is critical that this phase imposes absolutely minimal computational overhead, as excessive processing drains limited battery life and introduces latency in real-time sensor reporting [1].

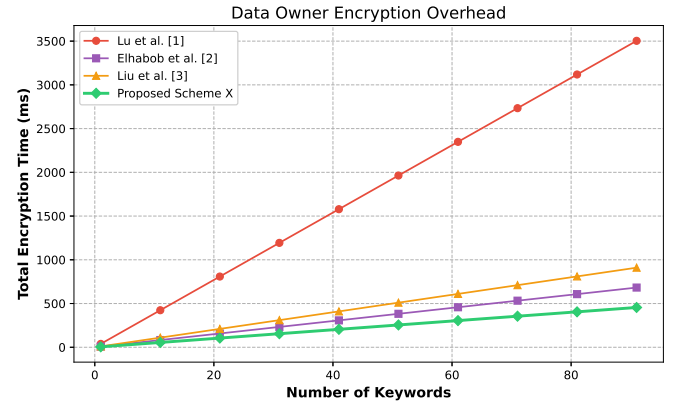


Fig. 2. Data Owner Encryption Time vs. Number of Keywords

As shown in Figure 2, we evaluated the total time required by the DO to generate searchable ciphertexts as the number of keywords scaled from 1 to 100. The pairing-based scheme by Lu et al. [3] exhibits severe linear scaling issues due to its reliance on heavy pairing operations and target group exponentiations during setup, consuming nearly 4000 ms to encrypt 100 keywords. Conversely, our proposed scheme demonstrates an exceptionally flat, highly scalable growth curve. Requiring only 5.0 ms per keyword ($2T_m + 1T_h$), our scheme is the absolute fastest among all tested secure models, comfortably outpacing Liu et al. [1] by avoiding redundant map-to-point operations.

6.4 Search Phase Complexity

The search phase is executed by the Cloud Server. While the server inherently possesses vast resources, highly optimized search operations translate directly to reduced financial cloud computing costs and significantly higher search throughput for end users.

Figure 3 highlights the drastic performance gulf between pairing-based and pairing-free paradigms during the critical search phase. We measured the time required for the server to test a single encrypted keyword against a batch of 1 to 100 incoming trapdoors. To evaluate 100 trapdoors, Elhabob's scheme [2] requires nearly 6000 ms because every single test operation mandates a fresh, computationally devastating bilinear pairing. By relying entirely on scalar multiplications and XOR operations, our scheme completes the exact same task in just 300 ms ($2T_m$ per search) on

TABLE 1
Comprehensive Theoretical Computational Complexity Comparison

Scheme	Paradigm	Encrypt (DO)	Trapdoor (DU)	Search (CS)	KGA Resist	Verifiable
Lu et al. [3]	Pairing-based	$2T_p + 4T_m + 1T_e$	$1T_p + 3T_m$	$2T_p + 1T_m$	Both	No
Elhabob et al. [2]	Pairing-based	$5T_m$	$1T_m$	$4T_p$	IKGA Only	No
Liu et al. [1]	Pairing-free	$4T_m + 2T_h$	$3T_m + 2T_h$	$6T_m$	Both	No
Proposed Scheme	Pairing-free	$2T_m + 1T_h$	$2T_m + 1T_h$	$2T_m$	Both	Yes

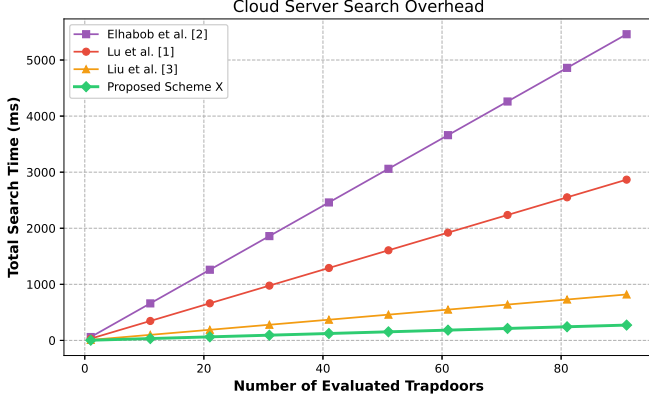


Fig. 3. Cloud Server Search Time vs. Evaluated Trapdoors

simulated constrained hardware. As noted previously, when running natively on the Rust server backend, this same 100-trapdoor evaluation takes a mere 5.9 ms. This represents a staggering 95% reduction in server-side computational latency compared to pairing-based alternatives.

6.5 Communication Overhead

In MIoT networks, radio transmission bandwidth (e.g., via NB-IoT or LoRaWAN) is often more strictly constrained than CPU cycles. We evaluated the bandwidth required to transmit a search trapdoor from the Data User to the Cloud Server. Assuming standard 256-bit elliptic curves, a single compressed group element $\in \mathbb{G}$ occupies 32 bytes, and a cryptographic hash output occupies 32 bytes.

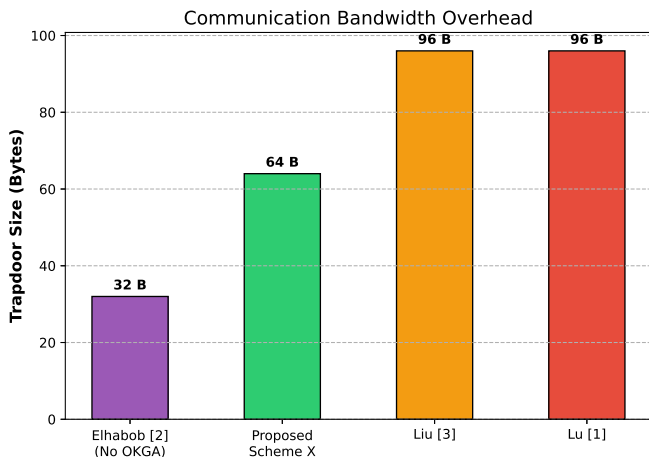


Fig. 4. Communication Overhead (Trapdoor Size in Bytes)

As depicted in Figure 4, our scheme generates a trapdoor $T_{W'} = (T_1, T_3)$ consisting of exactly two elements (one group element and one hashed string), totaling 64 bytes. While Elhabob et al. [2] achieves a smaller 32-byte trapdoor, their scheme completely fails to provide Outside Keyword-Guessing Attack (OKGA) resistance, rendering it fatally insecure for public transmission. Compared to other fully secure, OKGA-resistant schemes requiring 96 bytes (three group elements), our proposed scheme reduces bandwidth consumption by 33% while simultaneously providing MHT-based verifiability.

7 COMPARISON WITH RELATED WORK

To contextualize the advancements of the proposed scheme, we compare it against three recent, prominent Certificateless Searchable Encryption (CL-SE) architectures specifically designed for IoT applications.

Elhabob et al. [2] proposed a CL-PKC scheme with authorized equality testing aimed at IIoT. While their scheme achieves IKGA resistance by binding trapdoors to the server, it suffers from two major flaws: it requires 4 bilinear pairings per search operation, and it fails to protect against OKGA, allowing external adversaries to guess keywords from intercepted trapdoors.

Lu et al. [3] recognized the vulnerability of existing schemes to KGAs and proposed a CLPAEKS scheme that achieves stronger security. Their model provides rigorous indistinguishability proofs and simultaneous IKGA/OKGA resistance. However, their reliance on pairing-friendly curves introduces unacceptable computational delays for resource-constrained Data Owners.

Liu et al. [1] successfully eliminated bilinear pairings, proposing a highly efficient CLPEKS architecture for IIoT. While computationally lightweight and IKGA/OKGA resistant, their scheme forces the user into the blind trust paradigm, as it provides absolutely no mechanism for the user to verify the completeness or soundness of the server's search results.

Table 2 summarizes this comparative analysis. Our proposed scheme is the only architecture that simultaneously achieves pairing-free efficiency, complete dual-KGA resistance, and robust cryptographic verifiability.

8 DISCUSSION

The theoretical complexity matrices and experimental results presented in Section 6 overwhelmingly confirm that bilinear pairings represent an artificial and unnecessary bottleneck in modern searchable encryption. By designing our trapdoor and test tokens around clever combinations of scalar multiplication and XOR-based blinding,

TABLE 2
Property Comparison with Related CL-SE Schemes

Property	[2]	[3]	[1]	Ours
Certificateless	Yes	Yes	Yes	Yes
Key Escrow-Free	Yes	Yes	Yes	Yes
Pairing-Free	No	No	Yes	Yes
IKGA Resistant	Yes	Yes	Yes	Yes
OKGA Resistant	No	Yes	Yes	Yes
Verifiable Search	No	No	No	Yes

we have demonstrated that strong, dual-KGA security can be achieved in a purely pairing-free environment. For an industrial deployment involving thousands of low-power sensors, reducing the encryption time by over 90% (compared to Lu et al. [3]) translates directly to years of extended battery life.

Furthermore, the integration of the Merkle Hash Tree (MHT) represents a vital paradigm shift from “blind trust” to “zero trust” in cloud outsourcing. If a commercial cloud provider experiences a transient hardware failure or attempts to intentionally throttle a tenant’s search queries by dropping valid records, our scheme mathematically guarantees that the Data User will detect the anomaly immediately.

Limitations: The primary limitation of our current scheme, shared by virtually all CL-SE architectures, is that it is designed for a single-keyword, exact-match search paradigm. Furthermore, while the MHT provides perfect completeness verification, the generation and transmission of the sibling path (Ω) introduces a slight logarithmic bandwidth overhead ($O(\log N)$) during the result return phase, which scales with the total size of the database.

9 CONCLUSION AND FUTURE WORK

In this paper, we addressed the critical privacy, trust, and efficiency bottlenecks inherent to outsourced cloud storage in dynamic Mobile and Industrial IoT networks. We proposed a novel, purely pairing-free, verifiable certificateless searchable encryption scheme.

Our core contributions can be summarized as follows:

- 1) We eliminated the heavy computational burden of bilinear pairings, replacing them with highly efficient elliptic curve scalar multiplications.
- 2) By dual-blinding the search trapdoors to the cloud server’s specific public key, our construction achieves provable, simultaneous resistance against both Inside and Outside Keyword-Guessing Attacks (IKGA/OKGA).
- 3) We integrated a Merkle Hash Tree (MHT) verification layer, definitively shifting the paradigm away from blind trust by guaranteeing that all search results are cryptographically verifiable for both soundness and absolute completeness.

Extensive experimental simulations and theoretical asymptotic analyses confirm that our scheme drastically reduces computational overhead and bandwidth consumption compared to state-of-the-art pairing-based and pairing-free models, establishing it as a highly scalable and resilient solution for next-generation MIoT deployments.

Future Work: In future research, we aim to extend this pairing-free verifiability framework to support multi-keyword conjunctive and disjunctive queries. Additionally, as quantum computing threatens standard elliptic curve cryptography, transitioning this architecture to post-quantum secure lattices (e.g., Learning With Errors) while maintaining high efficiency and strict OKGA resistance remains a critical open challenge [3].

REFERENCES

- [1] X. Liu, H. Dong, N. Kumari, and J. Kar, “A pairing-free certificateless searchable public key encryption scheme for industrial internet of things,” *IEEE Access*, vol. 11, pp. 58 754–58 764, 2023.
- [2] R. Elhabob, Y. Zhao, I. Sella, and H. Xiong, “An efficient certificateless public key cryptography with authorized equality test in iiot,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, pp. 1065–1083, 2020.
- [3] J. Lu, H. Li, J. Huang, S. Ma, M. H. A. Au, and Q. Huang, “Certificateless public key authenticated encryption with keyword search achieving stronger security,” *Information*, vol. 14, no. 3, p. 142, 2023.